```
#include <iostream>
#include <cstring>
using namespace std;

class Stack
{
private:
   struct Node
   {
      char data;
      Node *next;
   };

   Node *top;

public:
   Stack()
   {
      top = NULL;
   }

   ~Stack()
   {
      while (top != NULL)
      {
         Node *temp = top;
         top = top->next;
         delete temp;
      }
   }

   void push(char x)
   {
      Node *newnode = new Node;
      newnode->data = x;
```

```
    newnode->next = top;
    top = newnode;
}

char pop()
{
    if (top == NULL)
        return '\0';

    Node *temp = top;
    char x = temp->data;
    top = top->next;
    delete temp;
    return x;
}

char peek()
{
    if (top == NULL)
        return '\0';
    return top->data;
}

int precedence(char op)
{
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return 0;
}

void postfix()
{
    char exp[100];
```

```
char post[100];
int j = 0;

cout << "\nEnter infix expression: ";
cin >> exp;

int length = strlen(exp);

for (int i = 0; i < length; i++)
{
   char ch = exp[i];

   // Operand
   if (isalnum(ch))
   {
      post[j++] = ch;
   }

   // Left parenthesis
   else if (ch == '(')
   {
      push(ch);
   }

   // Right parenthesis
   else if (ch == ')')
   {
      while (peek() != '(')
      {
         post[j++] = pop();
      }
      pop(); // remove '('
   }

   // Operator
```

```
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        {
            while (top != NULL &&
                   precedence(peek()) >= precedence(ch))
            {
                post[j++] = pop();
            }

            push(ch);
        }
    }

    // Pop remaining operators
    while (top != NULL)
    {
        post[j++] = pop();
    }

    post[j] = '\0';

    cout << "\nPostfix expression: " << post << endl;
  }
};

int main()
{
  int ch;
  Stack st;

  do
  {
    cout << "\n1. Infix to Postfix";
    cout << "\n2. Exit";
    cout << "\nEnter your choice: ";
    cin >> ch;
```

```
        switch (ch)
        {
        case 1:
            st.postfix();
            break;

        case 2:
            cout << "Exiting...\n";
            break;

        default:
            cout << "Invalid choice\n";
        }

    } while (ch != 2);

    return 0;
}
```

FILE NAME

infix_to_postfix.cpp

COMMAND

g++ infix_to_postfix.cpp -o infix

./infix